BATORE PERSONAL ASSISTANT

Product, Architecture & Implementation Specification

You are building an internal-use AI personal assistant for Batore.

This application is not intended to be a generic SaaS product at this stage. It is being built specifically for internal use by the Batore team and should prioritize usefulness, speed, contextual understanding, and low-friction interaction over generic productivity-app conventions.

The product should feel like a human executive/personal assistant that exists inside a conversational interface.

The user should be able to tell it things naturally.

They should not have to manage the assistant through forms, task lists, calendars, tags, databases, or complicated workflows.

The assistant should understand what the user says, remember it, connect it to existing context, ask questions when necessary, identify conflicts, track commitments, and execute explicit instructions.

1. THE CORE PRODUCT IDEA

The fundamental interaction is:
The user talks. The assistant understands and manages the resulting state.

The user may enter:

- free-form text
- screenshots
- photographs
- PDFs
- documents
- eventually voice
- eventually emails/messages forwarded into the system

Examples:

> *"Barkha needs to give me the article by 6."*
>
> *"Remind me at 5 to ask her if she hasn't sent it."*
>
> *"Barkha gave the article by 11."*
>
> *"She had a family emergency."*
>
> *"I finished the Hult poster."*
>
> *"Arun is handling the backend for the CRM."*
>
> *"Arun hasn't sent the schema yet."*
>
> *"I have a meeting with Karthik at 5 tomorrow."*
>
> *"Schedule Arun at 5 tomorrow."*

The assistant should interpret these statements and maintain a persistent model of the user's world.

2. THIS IS NOT A PRODUCTIVITY APP

Do NOT build this as:

- Todoist with AI
- Notion with AI
- Google Calendar with AI
- a chatbot with a task database
- a generic "AI productivity assistant"

The user should NOT have to:

- create a task manually
- select a category
- select a project
- choose a priority
- open a calendar
- mark a task complete
- manually create a contact
- manually create a reminder
- organize information into folders

The assistant should infer these structures from natural conversation.

The interface may eventually expose these structures for inspection, but they should not be required for normal interaction.

3. THE CENTRAL PRODUCT PRINCIPLE

Use this as the highest-level product rule:
Don't make the user organize their life for the assistant. Make the assistant understand their life.

The user should be able to dump information into the system exactly as they would tell a human assistant.

The assistant converts unstructured human communication into structured state.

4. THE ASSISTANT IS NOT A LIFE COACH

This is extremely important.

The assistant should NOT give unsolicited productivity advice.

If the user says:

> *"I have an inflammation exam next Friday."*

The assistant should log the exam.

It should NOT respond:

> *"Here's a seven-day study plan."*

If the user says:

> *"I haven't studied inflammation."*

It should store that context.

It should NOT automatically lecture the user or create a study schedule.

Only provide planning/advice when the user explicitly asks for it.

For example:

> *"I have an inflammation exam next Friday. Help me plan."*

Now planning is appropriate.

The assistant manages the user's information and commitments.

It does not attempt to manage the user's personality or life unless asked.

5. INFORMATION VS ACTION

The assistant must distinguish between statements that provide information and statements that request action.

INFORMATION

User:

> *"Barkha needs to give me the article by 6."*

The assistant should understand and record the commitment.

ACTION

User:

> *"Remind me at 5 to ask Barkha."*

Create a reminder.

COMPLETION UPDATE

User:

> *"Barkha gave the article at 11."*

Update the existing commitment.

CONTEXT

User:

> *"She had a family emergency."*

Attach contextual information to the relevant commitment/event.

QUESTION

User:

> *"What happened with Barkha's article?"*

Retrieve the relevant history.

EXECUTION

User:

> *"Send Arun this."*

Execute through an authorized integration if available.

Do not confuse these modes.

6. COMMITMENTS ARE A CORE DATA MODEL

Do NOT make "Task" the primary abstraction.

Use a broader concept:
COMMITMENT

A commitment represents:
Who is expected to do what, for whom, by when, and what eventually happened.

This is fundamental to the application.

Examples:
Someone owes the user something

> *"Barkha needs to give me the article by 6."*

- Owner: Barkha
- Recipient: User
- Object: Article
- Expected: 6:00 PM
- Status: Pending

User owes someone something

> *"I need to send Hult the final poster by Wednesday."*

- Owner: User
- Recipient: Hult
- Object: Final poster
- Expected: Wednesday
- Status: Pending

User completes something

> *"I finished the Hult poster."*

- Owner: User
- Object: Hult poster
- Status: Completed
- Completed at: timestamp

Someone completes something late

> *"Barkha gave the article at 11."*

If expected time was 6 PM:

- Expected: 6 PM
- Completed: 11 PM
- Status: Completed late
- Delay: 5 hours

The assistant should understand that the event represents a change in state rather than creating an unrelated new note.

7. OWNERSHIP MATTERS

The assistant must distinguish who owes whom.

These statements are fundamentally different:

> *"I need to send Barkha the article by 6."*

versus:

> *"Barkha needs to send me the article by 6."*

The underlying commitment should explicitly represent:

- owner
- recipient
- action/object
- expected time
- status

This allows the assistant to track both:

Things the user owes other people

and

Things other people owe the user

This is one of the major differentiating features of the product.

8. COMMITMENT LIFECYCLE

A commitment can move through states such as:

- Pending
- In progress
- Waiting
- Waiting on someone
- Due soon
- Overdue
- Completed
- Completed late
- Cancelled
- Blocked
- Superseded

Do not require the user to manually select these states.

Infer state from conversation and system events.

Example:

> *"Arun hasn't sent the schema."*

Waiting on Arun.

If deadline passes:
Overdue / waiting on Arun.

If Arun sends it:

> *"Arun sent the schema."*

Completed.

9. PEOPLE ARE FIRST-CLASS ENTITIES

The assistant must understand people over time.

Example:
Barkha

- associated with MTTN
- frequently coordinates interviews
- has specific commitments
- has conversations
- is connected to projects
- has a history of interactions with the user

Arun

- associated with Batore
- handles backend
- involved in CRM

- has outstanding commitments

The assistant should build relationships from explicit information.

10. ASK ABOUT PEOPLE WHEN NECESSARY

When a person is first introduced, the assistant can ask a short contextual question when that information is useful.

Example:

User:

> *"Karthik needs to send me the poster tomorrow."*

A:

> *"Karthik from Hult?"*

User:

> *"Yeah."*

Now associate Karthik with Hult.

However, do not interrogate the user unnecessarily.

If context is obvious, infer it.

The principle is:
Ask when the answer materially improves future understanding. Don't ask just because you can.

11. AMBIGUITY HANDLING

The assistant should ask clarification questions when ambiguity could cause an incorrect action or memory.

Example:

User:

> *"Send it to her tomorrow."*

A:

> *"Who do you mean by 'her'?"*

If there are two Aruns:

User:

> *"Remind me to ask Arun about the poster."*

A:

> *"Arun from Batore or Arun from MTTN?"*

If the context is unambiguous, do not ask.

12. INFERENCE LEVELS

Internally distinguish between:
CONFIRMED

Explicitly stated by the user.

INFERRED

Highly probable based on repeated context.

UNCERTAIN

Insufficient information.

Never convert an uncertain inference into a confirmed fact without appropriate confirmation.

Example:
If the user repeatedly mentions Arun in relation to Batore backend work, the system may infer the association.

But if there are multiple Aruns, ask.

13. THE ASSISTANT SHOULD UNDERSTAND THE USER OVER TIME

As the system accumulates information, it should build a useful model of:

- projects
- collaborators
- recurring organizations
- commitments
- deadlines
- communication patterns
- explicit preferences
- recurring workflows
- relationships
- working context

However, avoid amateur psychological profiling.

Do not make unsupported claims like:

> *"You procrastinate because you fear failure."*

Instead use observable facts:

> *"You've postponed this three times."*

The assistant can eventually understand patterns in the user's work, but should remain grounded in actual evidence.

14. MEMORY MUST BE STRUCTURED

Do NOT treat memory as simply an enormous chat transcript.

Use structured entities plus semantic memory.

Core entities:

- User
- Person
- Organization
- Project
- Commitment
- Event
- Reminder
- Conversation
- Message
- Document/File
- Relationship
- Workflow/Conditional rule
- Memory

15. RELATIONSHIP GRAPH

The system should conceptually maintain a graph such as:

```
User
 ■■■ works on → Batore
 ■       ■■■ Arun → Backend
 ■       ■■■ Karthik → CRM
 ■       ■■■ Project X
 ■
 ■■■ works with → MTTN
 ■       ■■■ Barkha
 ■       ■■■ Interview Candidate
 ■
 ■■■ works with → Hult
         ■■■ Karthik
```

Relationships should have:

- type
- source
- confidence
- created_at
- updated_at
- optional expiration

- historical versions

This allows the assistant to understand context rather than merely retrieve keywords.

16. MEMORY PROVENANCE

Important memories should have provenance.

Example:
Fact: Arun handles Batore backend.

Source: Conversation from September 7, 2026.

Confidence: Confirmed.

This allows the user to correct the assistant.

17. MEMORY CORRECTION

The user must be able to correct stored information conversationally.

Example:

A:

> *"Arun handles the backend."*

User:

> *"No, Karthik handles it now."*

The system should:
Update the current relationship. Preserve relevant historical information. Mark Arun's previous role as historical if appropriate. Associate Karthik with the current role.

Do not simply append contradictory facts forever.

The memory system must understand temporal changes.

18. TEMPORAL UNDERSTANDING

The assistant must understand:

- today
- tomorrow
- yesterday
- tonight
- this evening
- next Friday

- this week
- by Wednesday
- before the meeting
- after Arun replies
- in two hours
- later

Normalize timestamps internally.

Use the user's timezone.

Do not rely on vague text when scheduling actual actions.

19. REMINDERS

Reminders should be conversational.

User:

> *"Barkha needs to give the article by 6. Remind me at 5 to ask her."*

Create:

- Commitment:
- Barkha → article → user
- Expected: 6 PM

Reminder: 5 PM Action: Ask Barkha if article has not arrived.

At 5 PM:

> *"Barkha's article is due at 6 PM. Ask her if you haven't received it."*

If the user then says:

> *"Barkha gave it at 11."*

Update the commitment.

The user should NOT need to manually open a task or reminder screen.

20. CONTEXT AFTER COMPLETION

This is important.

If the user says:

> *"Barkha gave the article by 11."*

The assistant should recognize:

- commitment existed
- expected time was 6 PM
- actual completion was 11 PM

- commitment was completed late

It may optionally ask:

> *"Got it — she sent it at 11 PM, five hours late. Do you want to add any context for the delay?"*

This should be OPTIONAL.

If the user says:

> *"She had a family emergency."*

Attach that context.

If the user says:

> *"No."*

Do not continue asking.

Do not make judgmental statements about Barkha.

The system is recording context, not passing moral judgment.

21. COMPLETING THE USER'S OWN WORK

The user should be able to complete something simply by saying it.

User:

> *"Finished the Hult poster."*

The assistant should update the relevant commitment/task.

No need to open:
Projects → Hult → Tasks → Poster → Mark Complete

The conversational interface IS the control interface.

22. CONVERSATIONAL STATE UPDATES

Every user message should potentially modify state.

Example:

User:

> *"Arun still hasn't sent the schema."*

This should update:

- Commitment:
- Arun → schema → User
- Status:
- Waiting / overdue depending on deadline
- Last observed:

- Not received
- Updated:
- Current timestamp

The assistant should not create duplicate commitments every time the user mentions them.

Use entity resolution and semantic matching.

23. DUPLICATE DETECTION

If the user says:

> *"I need to finish the poster."*

Then later:

> *"Still need to finish that Hult poster."*

Do not create two independent tasks.

Resolve them to the same underlying commitment where confidence is sufficient.

24. CONFLICT DETECTION

The assistant should detect meaningful conflicts.

Example:

Existing:
5 PM — Hult meeting.

User:

> *"Schedule Arun at 5 tomorrow."*

A:

> *"5 PM tomorrow conflicts with your Hult meeting. Should I move the call or keep both?"*

Do not automatically choose.

Other conflicts:

- overlapping meetings
- impossible deadlines
- contradictory commitments
- duplicate events
- contradictory people information
- dependency conflicts

The assistant should surface important conflicts without micromanaging.

25. CONDITIONAL COMMITMENTS / WORKFLOWS

Support natural conditional instructions.

Example:

> *"If Arun hasn't sent the schema by Friday, remind me."*

Represent:

- Condition:
- Schema not received from Arun
- Deadline:
- Friday
- Action:
- Remind user

Another:

> *"If Barkha doesn't reply by tomorrow, ask me whether I want to follow up."*

Another:

> *"Let me know if the venue isn't confirmed by Wednesday."*

These are persistent rules.

The assistant should evaluate them automatically.

For MVP, focus on notifications/reminders.

External autonomous actions can come later.

26. PROACTIVE BEHAVIOUR

The assistant should be proactive, but not annoying.

Good:

> *"Barkha's article is five hours overdue and you haven't marked it as received."*

Good:

> *"You have a 5 PM meeting with Hult and you're trying to schedule Arun at the same time."*

Bad:

> *"You have 17 tasks! Here's how to optimize your day!"*

Bad:

> *"You should study now."*

Bad:

> *"Would you like me to create a plan?" after every statement.*

Proactivity should be driven by actual relevance.

27. THE ASSISTANT SHOULD NOT OVER-ASK

Avoid confirmation fatigue.

If the user says:

> *"Finished the poster."*

Do not ask:

> *"Would you like me to mark the poster as complete?"*

Just do it.

If the user says:

> *"Remind me at 6."*

and there is only one obvious pending item:
Infer it if safe.

If there are multiple possible things:
Ask.

Use this principle:
Infer when safe. Ask when necessary. Never guess when guessing can cause a meaningful mistake.

28. USER CONTROL

The user should always be able to inspect and correct what the assistant believes.

Potential commands:

- "What do you know about Arun?"
- "What am I waiting on?"
- "What does Barkha owe me?"
- "What do I owe Hult?"
- "Forget that Arun works on backend."
- "That's not Karthik from Hult."
- "Show me everything pending for Batore."

The system should retrieve structured state.

29. THE PRIMARY UI

The UI should be intentionally minimal.

The primary experience should be a conversational space.

Do NOT make a giant dashboard the home screen.

Optional secondary views may exist:

- Today
- Commitments
- People
- Projects
- Memory
- Activity

But these should be inspection surfaces, not mandatory workflows.

The user should be able to operate the entire application conversationally.

30. CONVERSATION DESIGN

The assistant should feel like a competent human assistant.

Prefer "Got it." over "I've successfully analyzed your input and updated the commitment database."

Prefer "Karthik from Hult?" over "Please clarify which Karthik entity you are referring to."

Prefer "5 PM conflicts with your Hult meeting." over "A scheduling conflict has been detected."

The system can be technically sophisticated underneath while remaining conversational on the surface.

31. RESPONSE LENGTH

Default to concise responses.

The assistant should not repeat everything it understood unless useful.

Example:

User:

> *"I finished the Hult poster."*

A:

> *"Got it — Hult poster marked complete."*

Not a paragraph.

When clarification or reasoning is necessary, provide enough context to make the decision.

32. IMAGE INPUT

Support image input in the architecture.

Potential inputs:

- screenshots
- WhatsApp screenshots
- event posters
- documents
- photographs
- handwritten notes
- schedules
- design briefs

The system should extract useful information.

Example:
User uploads an event poster.

The assistant identifies:
- event name
- date
- time
- venue
- organizers
- people
- deadlines

But it should NOT automatically create a calendar event merely because an event was detected.

Interpret first. Act only when appropriate.

33. DOCUMENTS

Eventually support:
- PDF
- DOCX
- XLSX
- TXT
- images

Documents should be associated with relevant projects, people, and commitments.

Example:

User:

> *"This is the final Hult brief."*

Uploads PDF.

The system associates it with:
Hult → relevant project → brief.

34. EXTERNAL INTEGRATIONS

The architecture should eventually support:

- Gmail
- Google Calendar
- Google Drive
- messaging platforms where APIs permit
- internal Batore systems
- web search
- documents
- spreadsheets
- file storage

Potential actions:

- "Draft an email to Arun."
- "Send this to Arun."
- "Check if Arun replied."
- "Create a meeting."
- "Find the file Karthik sent."
- "Create a document from this."

External actions must have proper permissions.

Do not allow an LLM to arbitrarily perform consequential actions.

35. PERMISSION MODEL

Separate internal state changes from external actions.

Internal state changes (generally low risk):

- save memory
- update commitment
- create reminder
- update project
- store conversation

External actions (higher risk):

- send email
- send message
- modify calendar
- delete file

- publish something
- submit something

Build a permission system that supports:

- one-time confirmation
- persistent permission
- permission by action type
- permission revocation

For example:

> *"Always allow calendar creation."*
>
> *"Require confirmation before sending emails."*

36. TECHNICAL ARCHITECTURE

Use a modular architecture: Conversation UI → API/Backend → Assistant Orchestrator, which coordinates the LLM/AI layer, the Memory Engine, and Tools/Actions. These converge on a State Layer composed of a relational database, semantic memory, and an event log.

37. LLM SHOULD NOT DIRECTLY MODIFY DATABASE

Do not allow arbitrary model-generated database mutations.

Use structured tool/function calls.

Example:

```
{
  "action": "update_commitment",
  "commitment_id": "...",
  "changes": {
    "status": "completed",
    "completed_at": "..."
  },
  "confidence": 0.97
}
```

Validate the structured output.

Then execute the mutation through application code.

The LLM should propose state changes. The backend should validate and commit them.

38. MEMORY ARCHITECTURE

Use a hybrid architecture.

Relational database, for deterministic state:

- users
- people
- organizations
- projects
- commitments
- reminders
- events
- relationships
- workflows
- permissions
- messages
- action logs

Semantic/vector memory for contextual recall and unstructured understanding, layered on top of the relational core.